

Important Problems

First: Sequences:

Sequence Type	Formula
Natural Numbers	$S_n = \frac{n(n+1)}{2}$
Odd Numbers	$S_n = n^2$
Even Numbers	$S_n = n(n+1)$
Squares of Natural Numbers	$S_n = \frac{n(n+1)(2n+1)}{6}$
Cubes of Natural Numbers	$S_n = \left(\frac{n(n+1)}{2}\right)^2$
Arithmetic Progression (AP)	$S_n = \frac{n}{2}(2a + (n-1)d)$

1. Sum of First n Natural Numbers:

```
def sum_natural(n):  
    return n * (n + 1) // 2  
  
print(sum_natural(100)) # Example: Sum of first 100 number  
s
```

2. Sum of First n Odd Numbers:

```
def sum_odds(n):  
    return n ** 2  
  
print(sum_odds(500000000)) # Sum of first 500,000,000 odd  
numbers
```

3. Sum of First n Even Numbers:

```
def sum_evens(n):  
    return n * (n + 1)
```

```
print(sum_evens(500000000)) # Sum of first 500,000,000 even numbers
```

4. Sum of Squares of First n Natural Numbers:

```
def sum_squares(n):  
    return n * (n + 1) * (2 * n + 1) // 6  
  
print(sum_squares(100)) # Sum of squares of first 100 numbers
```

5. Sum of Cubes of First n Natural Numbers:

```
def sum_cubes(n):  
    return (n * (n + 1) // 2) ** 2  
  
print(sum_cubes(100)) # Sum of cubes of first 100 numbers
```

6. Sum of an Arithmetic Progression (AP)

```
def sum_ap(a, d, n):  
    return n * (2 * a + (n - 1) * d) // 2  
  
print(sum_ap(1, 2, 100)) # Sum of first 100 odd numbers  
                           (1, 3, 5, ...)  
# a: first_number, d: difference, n: number_of_numbers
```

Second: Problems:

1. Detecting Even or Odd Number

- Check whether a number is even or odd using modulo (%).

```
def is_even(n):  
    return n % 2 == 0
```

2. Detecting Prime Number

- Check whether a number is prime by checking divisibility from 2 to `sqrt(n)`.

```
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

3. Detecting Palindrome Number

- Check if a number reads the same forward and backward.

```
def is_palindrome(n):
    return str(n) == str(n)[::-1]
```

4. Finding Factorial of a Number

- Compute the factorial of a given number using recursion or iteration.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

5. Fibonacci Sequence

- Generate the `n`th Fibonacci number using recursion or iteration.

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

6. Sum of Digits in a Number

- Calculate the sum of the digits in a number.

```
def sum_of_digits(n):  
    sum = 0  
    for digit in str(n):  
        sum += int(digit)  
    return sum
```

7. Checking Armstrong Number

- Check if a number is an Armstrong number, where the sum of its digits raised to the power of the number of digits equals the number itself.

```
def is_armstrong(n):  
    num_str = str(n)  
    power = len(num_str)  
    return sum(int(digit) ** power for digit in num_str)  
    == n
```